

Class 14: Spark SQL & Data Sources

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

ED1, 406A, amitkdhar@iitbhlai

February 19, 2026

Agenda

- Spark SQL Interfaces
- The DataFrameReader and DataFrameWriter APIs
- Built-in Data Sources and File Formats
 - Parquet, ORC, JSON, CSV, Avro, Images, Binary
- Managed vs Unmanaged Tables
- Interacting with External Data Sources (JDBC)

- Spark SQL is the foundation for the DataFrame API.
- It allows interacting with structured data using SQL or the Dataset/DataFrame API.
- Key Capabilities:
 - Unifies Spark components (SQL, Streaming, MLlib).
 - Connects to Hive metastore.
 - Reads/Writes structured data (JSON, CSV, Parquet, etc.).
 - JDBC/ODBC connectivity.

The SparkSession

- The entry point for all Spark SQL functionality.
- Used to run SQL queries, create DataFrames, and manage tables.

Scala

```
1 val spark = SparkSession
2   .builder
3   .appName("SparkSQLExample")
4   .getOrCreate()
5
```

Python

```
1 from pyspark.sql import SparkSession
2 spark = (SparkSession
3   .builder
4   .appName("SparkSQLExample")
5   .getOrCreate())
6
```

Executing SQL Queries

- Use `spark.sql("...")` to execute ANSI SQL queries.
- Returns a `DataFrame`.

Scala

```
1 spark.sql("SELECT * FROM myTable")
2   .show()
3
```

Python

```
1 spark.sql("SELECT * FROM myTable") \
2   .show()
3
```

Temporary Views

- To query a DataFrame with SQL, you must register it as a view.
- `createOrReplaceTempView`: Session-scoped. Disappears when session ends.

Scala

```
1 val df = spark.read.json("data.json")
2 df.createOrReplaceTempView("people")
3 spark.sql("SELECT * FROM people WHERE
   age > 20")
4
```

Python

```
1 df = spark.read.json("data.json")
2 df.createOrReplaceTempView("people")
3 spark.sql("SELECT * FROM people WHERE
   age > 20")
4
```

Global Temporary Views

- `createOrReplaceGlobalTempView`: Shared across all sessions in the application.
- Kept in the `global_temp` database.
- Persists until the Spark application terminates.

Scala

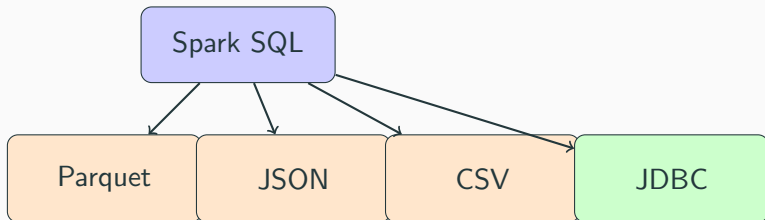
```
1 df.createOrReplaceGlobalTempView("g_
   people")
2 spark.sql("SELECT * FROM global_temp.g_
   people")
3
```

Python

```
1 df.createOrReplaceGlobalTempView("
   g_people")
2 spark.sql("SELECT * FROM global_temp.
   g_people")
3
```

Data Sources API

- Unified API for reading and writing data.
- **DataFrameReader**: Used to read data into a DataFrame.
- **DataFrameWriter**: Used to write a DataFrame to storage.
- Supports built-in formats (Parquet, JSON, CSV, etc.) and external plugins (JDBC, Kafka, Cassandra).




```
spark.read.format(...).option(...).schema(...).load()
```

- **format**: Specifies file format (e.g., "parquet", "csv"). Default is Parquet.
- **option**: Key-value configurations (e.g., "header" -> "true").
- **schema**: Optional explicit schema (StructType or DDL string).
- **load**: Path to the data source.

DataFrameReader Example

Scala

```
1 val df = spark.read
2   .format("csv")
3   .option("header", "true")
4   .option("mode", "FAILFAST")
5   .schema("id INT, name STRING")
6   .load("/path/to/data.csv")
7
```

Python

```
1 df = (spark.read
2   .format("csv")
3   .option("header", "true")
4   .option("mode", "FAILFAST")
5   .schema("id INT, name STRING")
6   .load("/path/to/data.csv"))
7
```

```
df.write.format(...).option(...).mode(...).save()
```

- **format**: Output format.
- **option**: Configuration (e.g., compression).
- **mode**: How to handle existing data.
- **save**: Destination path.

Writer Save Modes

Mode	Description
<code>append</code>	Append data to existing files.
<code>overwrite</code>	Overwrite existing data.
<code>errorIfExists</code>	Throw an error if data exists (default).
<code>ignore</code>	Do nothing if data exists.

DataFrameWriter Example

Scala

```
1 df.write
2   .format("parquet")
3   .mode("overwrite")
4   .option("compression", "snappy")
5   .save("/path/to/output")
6
```

Python

```
1 (df.write
2   .format("parquet")
3   .mode("overwrite")
4   .option("compression", "snappy")
5   .save("/path/to/output"))
6
```

- Open-source columnar storage format.
- **Default** format in Spark.
- Advantages:
 - Highly efficient compression (Snappy, Gzip).
 - Column pruning (read only needed columns).
 - Predicate pushdown (skip data chunks based on stats).
 - Preserves schema.

Row vs Columnar Storage

Row-Oriented (CSV, Avro)

ID	Name	Age
1	Jules	30
2	Brooke	25

Column-Oriented (Parquet)

1, 2, ... (IDs)
Jules, Brooke, ...
30, 25, ... (Ages)

Efficient for aggregations (e.g., Avg Age)

Working with Parquet

- Schema is embedded in the footer; no need to specify it manually.

Scala

```
1 // Reading
2 val df = spark.read.parquet("file.
   parquet")
3
4 // Writing
5 df.write
6   .mode("overwrite")
7   .parquet("output_folder")
8
```

Python

```
1 # Reading
2 df = spark.read.parquet("file.parquet")
3
4 # Writing
5 df.write \
6   .mode("overwrite") \
7   .parquet("output_folder")
8
```


- Common for semi-structured data.
- Spark supports two modes:
 - **Single-line** (default): Each line is a complete JSON object.
 - **Multi-line**: Standard JSON file (use `option("multiLine", true)`).
- Schema inference can be expensive; better to specify schema if known.

Reading JSON

Scala

```
1 val df = spark.read
2   .format("json")
3   .option("multiLine", true)
4   .load("data.json")
5
```

Python

```
1 df = spark.read \
2   .format("json") \
3   .option("multiLine", True) \
4   .load("data.json")
5
```

Note: If no schema is provided, Spark scans the whole file to infer it.

- Common for data exchange, spreadsheets.
- Key Options:
 - `header`: true/false (First line is header).
 - `inferSchema`: true/false (Guess types).
 - `delimiter`: Default is comma, can be changed.
 - `mode`: PERMISSIVE, DROPMALFORMED, FAILFAST.

Reading CSV

Scala

```
1 val df = spark.read
2   .option("header", "true")
3   .option("inferSchema", "true")
4   .csv("data.csv")
5
```

Python

```
1 df = spark.read \
2   .option("header", "true") \
3   .option("inferSchema", "true") \
4   .csv("data.csv")
5
```

- Row-based binary format.
- Schema stored in JSON format within the file.
- Efficient for serialization and schema evolution.
- Popular in Kafka and Hadoop ecosystems.
- Requires `format("avro")`. Since Spark 2.4, it is built-in.

Scala

```
1 val df = spark.read
2   .format("avro")
3   .load("data.avro")
4
5 df.write.format("avro")
6   .save("output.avro")
7
```

Python

```
1 df = spark.read \
2   .format("avro") \
3   .load("data.avro")
4
5 df.write.format("avro") \
6   .save("output.avro")
7
```

- Optimized Row Columnar.
- Similar to Parquet (columnar, efficient compression).
- Highly optimized for Hive.
- Spark 2.x+ supports a vectorized ORC reader for performance.

Reading/Writing ORC

Scala

```
1 val df = spark.read
2   .format("orc")
3   .load("data.orc")
4
5 df.write.format("orc")
6   .save("output.orc")
7
```

Python

```
1 df = spark.read \
2   .format("orc") \
3   .load("data.orc")
4
5 df.write.format("orc") \
6   .save("output.orc")
7
```


Image Files

- Since Spark 2.4, supports loading images into DataFrames.
- Useful for Deep Learning pipelines (e.g., TensorFlow, PyTorch).
- Schema: origin, height, width, nChannels, mode, data.

```
1 # Python
2 imageDF = spark.read.format("image").load("path/to/images")
3 imageDF.printSchema()
4
```

Binary Files

- Since Spark 3.0.
- Reads entire file content as binary blobs.
- Useful for processing unstructured data (PDFs, audio, unknown formats).
- Columns: path, modificationTime, length, content.

```
1 // Scala
2 val binaryDF = spark.read.format("binaryFile")
3   .option("pathGlobFilter", "*.jpg")
4   .load("/path/to/files")
5
```

Format Comparison

Format	Type	Schema	Compression	Splittable
Parquet	Columnar	Embedded	Very High	Yes
ORC	Columnar	Embedded	Very High	Yes
Avro	Row	Embedded	High	Yes
JSON	Text	Inferred	Low	No*
CSV	Text	Inferred	Low	No*

* Splittable depending on compression codec (e.g. bzip2 is splittable, gzip is not).

Managed vs Unmanaged Tables

- **Managed Table:** Spark manages both metadata and data.
 - `saveAsTable("myTable")`
 - `DROP TABLE` deletes data and metadata.
 - Stored in `spark.sql.warehouse.dir` (default Hive warehouse).
- **Unmanaged (External) Table:** Spark manages only metadata.
 - `option("path", "/path/to/data").saveAsTable("myTable")`
 - `DROP TABLE` deletes only metadata; data remains.

Creating Managed Tables

Scala

```
1 df.write
2   .mode("overwrite")
3   .saveAsTable("managed_table")
4
```

Python

```
1 df.write \
2   .mode("overwrite") \
3   .saveAsTable("managed_table")
4
```

Creating Unmanaged Tables

Scala

```
1 df.write
2   .option("path", "/tmp/data")
3   .saveAsTable("external_table")
4
```

Python

```
1 df.write \
2   .option("path", "/tmp/data") \
3   .saveAsTable("external_table")
4
```

Caching SQL Tables

- Just like DataFrames, tables can be cached for performance.
- Useful for tables accessed multiple times.
- Uses `CACHE TABLE` syntax or API.

```
1 -- SQL
2 CACHE TABLE myTable;
3 CACHE LAZY TABLE myTable; -- Caches only when first used
4 UNCACHE TABLE myTable;
5
```

- Spark can read/write data from/to RDBMS (PostgreSQL, MySQL, Oracle) using JDBC.
- Requires JDBC driver jar on the classpath.
- Read returns a DataFrame; Write saves a DataFrame as a table.
- **Important:** Does not execute arbitrary SQL on DB (like UPDATE/DELETE). Focuses on data transfer.

Reading from JDBC

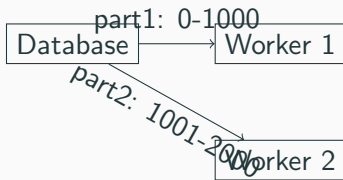
```
1 // Scala
2 val jdbcDF = spark.read
3   .format("jdbc")
4   .option("url", "jdbc:postgresql://host:port/db")
5   .option("dbtable", "schema.tablename")
6   .option("user", "username")
7   .option("password", "password")
8   .load()
9
```

Writing to JDBC

```
1 # Python
2 df.write \
3     .format("jdbc") \
4     .option("url", "jdbc:postgresql://host:port/db") \
5     .option("dbtable", "schema.tablename") \
6     .option("user", "username") \
7     .option("password", "password") \
8     .mode("append") \
9     .save()
10
```

Partitioning in JDBC

- By default, JDBC read runs on a single executor (1 partition).
- This is slow for large tables.
- **Solution:** Specify partitioning options to parallelize read.
 - `partitionColumn`, `lowerBound`, `upperBound`, `numPartitions`.



JDBC Partitioning Code

```
1 # Python
2 df = spark.read \
3     .format("jdbc") \
4     .option("url", url) \
5     .option("dbtable", "employees") \
6     .option("partitionColumn", "emp_id") \
7     .option("lowerBound", "1") \
8     .option("upperBound", "10000") \
9     .option("numPartitions", "10") \
10    .load()
11
```

- Ensure `postgresql-*.jar` is available.
- Run spark shell with `--jars postgresql-42.2.5.jar`.
- Supports pushdown of filters to the database (where applicable).

- Similar to Postgres.
- Connection URL: `jdbc:mysql://host:port/db`.
- Driver class: `com.mysql.jdbc.Driver`.

- URL: `jdbc:sqlserver://host:1433;databaseName=db`
- Driver: `com.microsoft.sqlserver.jdbc.SQLServerDriver`

Higher-Order Functions in Spark SQL

- Spark SQL supports complex types: Arrays, Maps.
- Higher-order functions allow manipulating arrays directly in SQL.
- Examples: `transform`, `filter`, `aggregate` (on arrays).

```
1 SELECT transform(array(1, 2, 3), x -> x + 1) as incremented;  
2 -- Result: [2, 3, 4]  
3
```


User-Defined Functions (UDFs)

- Extend Spark SQL with custom logic.
- Can be registered in Scala/Python and used in SQL.
- **Performance Note:** Python UDFs can be slower due to serialization; Scala UDFs are faster.

UDF Example

Scala

```
1 val cubed = (s: Long) => s * s * s
2 spark.udf.register("cubed", cubed)
3 spark.sql("SELECT cubed(2)")
4
```

Python

```
1 from pyspark.sql.types import LongType
2 def cubed(s): return s * s * s
3 spark.udf.register("cubed", cubed,
4                     LongType())
5 spark.sql("SELECT cubed(2)")
6
```

Bucketing

- Optimization technique for tables.
- Pre-shuffles and pre-sorts data into buckets on disk.
- Avoids expensive shuffles during subsequent Joins/Aggregations.
- Usage: `bucketBy(n, "col")`.

```
1 df.write
2   .bucketBy(8, "id")
3   .sortBy("id")
4   .saveAsTable("bucketed_table")
5
```

- **Spark SQL** unifies access to data via SQL and DataFrames.
- **DataFrameReader/Writer** provide a consistent API for various formats.
- **Parquet** is the default, efficient columnar format.
- **JDBC** allows connecting to external RDBMS; partitioning is key for performance.
- **Managed vs Unmanaged** tables dictate who controls the data lifecycle.

Questions?